



**EMBEDDED SYSTEM FINAL PROJECT REPORT
DEPARTMENT OF ELECTRICAL ENGINEERING
UNIVERSITAS INDONESIA**

SMART PLANT SYSTEM

Adaptive Soil Moisture Monitoring and Irrigation Control System
ATmega328P (Arduino Uno) — AVR Assembly Language

GROUP 21

MARSHAL AUFA D.	2406346913
ZAHIR	2406487084
SYIFA NAILA MAULIDYA	2406436940
M. RIFQI FADIL ITSNAIN	2406355306
ARYA WIBAWA ATMANEGARA	2406420431

PREFACE

Alhamdulillah, all praise be to God Almighty for His grace and guidance that enabled this group to complete the final project for the Embedded System (MBD) practicum at the Department of Electrical Engineering, Universitas Indonesia. This report presents Smart Plant System, an adaptive soil moisture monitoring and irrigation system based on the ATmega328P microcontroller and programmed entirely in AVR Assembly language. The project was developed to address the common problem of inconsistent plant watering, which often leads to overwatering or underwatering.

Unlike conventional automatic watering systems that use fixed-threshold relay control, Smart Plant System applies adaptive irrigation by combining soil moisture sensing, environmental monitoring through the BME280 sensor, and PWM-based pump control using MOSFET switching. The system dynamically adjusts watering intensity according to soil condition, temperature, and humidity, while providing precise low-level hardware control through AVR Assembly programming.

This project integrates all eight practicum modules, including ADC, Timer, USART, PWM, EEPROM, Interrupts, I/O Programming, and I2C communication. During development, the group also faced hardware challenges such as electromagnetic interference (EMI) from the DC pump motor, which affected ADC readings and serial communication. These experiences provided valuable insight into power management and mixed-signal embedded system design, and the group sincerely thanks the lecturers, teaching assistants, and collaborators for their guidance and support throughout the project.

Depok, Mei 15, 2026

TABLE OF CONTENTS

CHAPTER 1: INRODUCTION

- 1.1 BACKGROUND
- 1.2 PROJECT DESCRIPTION
- 1.3 OBJECTIVES
- 1.4 ROLES AND RESPONSIBILITIES

CHAPTER 2: IMPLEMENTATION

- 2.1 EQUIPMENT
- 2.2 SYSTEM ARCHITECTURE
- 2.3 HARDWARE IMPLEMENTATION
- 2.4 SOFTWARE IMPLEMENTATION

CHAPTER 3: TESTING AND ANALYSIS

- 3.1 TESTING METHODOLOGY
- 3.2 TEST RESULTS
- 3.3 ANALYSIS

CHAPTER 4: CONCLUSION

REFERENCES

APPENDICES

- APPENDIX A: PROJECT SCHEMATIC
- APPENDIX B: DOCUMENTATION PHOTOS
- APPENDIX C: AVR REGISTER CONFIGURATION SUMMARY
- APPENDIX D: DECISION LOGIC STATE TABLE

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

The agricultural and horticulture sector increasingly faces challenges related to water resource efficiency and labor-intensive monitoring of plant health. In both domestic and small-scale agricultural settings, over-watering and under-watering remain among the most common causes of plant mortality, leading to resource waste and economic loss. Research in precision agriculture consistently identifies inconsistent irrigation as the primary factor behind preventable plant death in household and urban farming contexts. The need for an intelligent, autonomous system capable of monitoring soil moisture conditions in real time and responding with proportional, environmentally-aware irrigation has become evident across contexts ranging from domestic houseplant care to greenhouse management.

Embedded systems offer a highly suitable platform to address this challenge. The ATmega328P microcontroller, which forms the computational core of the widely used Arduino Uno platform, provides sufficient computational resources, peripheral richness — including onboard ADC, TWI (I2C), USART, hardware timers with PWM outputs, external interrupt lines, and internal EEPROM — and low power consumption to serve as the brain of an intelligent plant care system. The microcontroller's 8-bit AVR architecture is well-documented and widely studied in electrical engineering curricula, making it an ideal platform for both educational exploration and practical embedded application development.

When programmed directly in AVR Assembly language rather than through high-level abstractions such as the Arduino core library, the ATmega328P can be driven with maximum efficiency, precise register-level control, and deterministic timing characteristics that are impossible to guarantee through compiler-generated code. This approach forces the developer to engage directly with hardware registers, interrupt vector tables, I2C protocol state machines, ADC timing constraints, and EEPROM write sequencing — deepening understanding of embedded system architecture in a way that high-level programming cannot provide.

Previous commercial solutions for automated irrigation typically rely on Wi-Fi-connected microcontrollers, cloud platforms, and mobile applications, introducing complexity, cost, dependency on external infrastructure, and single points of failure. A standalone

embedded system implemented entirely in Assembly language represents a lean, highly reliable, and educationally rigorous alternative: sophisticated automation achieved at the hardware level, without relying on operating systems, wireless connectivity, or external services.

The Smart Plant System project was therefore conceived as a practical, academically comprehensive final project that addresses a genuine real-world problem — the challenge of consistent, adaptive plant irrigation — while simultaneously demonstrating mastery of the embedded systems concepts covered throughout the MBD course curriculum: analog sensing through ADC, multi-sensor I2C communication, human-readable telemetry over USART, proportional actuator control through PWM, persistent user settings through EEPROM, non-blocking system scheduling through hardware timers, and event-driven user interaction through external interrupts. The result is a system that is not merely a collection of demonstration modules, but a cohesive product with genuine practical utility.

1.2 PROJECT DESCRIPTION

Smart Plant System is a fully automated, adaptive plant monitoring and irrigation system based on the ATmega328P microcontroller, programmed entirely in AVR Assembly language. The system is designed to operate autonomously without human intervention under normal conditions, while providing intuitive user controls for manual override and threshold customization.

The system's sensing pipeline begins with a capacitive soil moisture sensor connected to the ATmega328P's 10-bit ADC on channel A0. The sensor's analog voltage output, inversely proportional to soil moisture content, is sampled ten times per reading cycle and averaged to reduce electrical noise. A IIR smoothing filter further stabilizes the reading over time. The resulting moisture percentage, calibrated against empirically determined dry and wet ADC reference values, drives the core decision logic. In parallel, a BME280 multi-environment sensor communicates over the I2C bus (SDA/SCL on A4/A5) to provide ambient temperature, relative humidity, and atmospheric pressure readings.

The irrigation actuator is a DC mini water pump controlled through an IRLZ44N N-channel MOSFET in PWM switching configuration. Timer0 of the ATmega328P is configured in Fast PWM mode on OC0A (digital pin D6), generating a PWM signal at approximately 977

Hz. The duty cycle — and therefore the pump's flow rate — is determined by the adaptive decision logic: critically dry soil produces maximum pump speed (PWM = 120/255), moderately dry soil triggers medium speed (PWM = 100/255), and mildly dry soil activates low speed (PWM = 80/255). Additional environmental context from the BME280 provides a temperature-based PWM boost, increasing irrigation intensity on hot days. A hysteresis mechanism prevents rapid pump cycling at the threshold boundary.

User interaction is provided through two push buttons connected to external interrupt pins INT0 (D2) and INT1 (D3). The first button activates a manual override mode that forces the pump on at maximum duty cycle regardless of sensor readings. The second button reads the current position of a 10k Ω potentiometer (A1) and stores the resulting moisture threshold percentage to the ATmega328P's internal EEPROM at address 0x00, ensuring the setting persists across power cycles. Button state changes are processed in the main loop with a 300 ms debounce window.

System status and data are communicated through four output channels: a 16x2 LCD display connected to the I2C bus through a PCF8574 I2C expander (address 0x27) shows real-time soil moisture percentage, temperature, humidity, and pump status; USART serial communication at 9600 baud transmits a formatted telemetry block every second for PC monitoring; a green LED (D4) indicates normal standby status; a red LED (D5) signals active irrigation or critical condition; and an active buzzer (D7) provides audio alerts when soil moisture falls below the critical threshold of 20%.

The entire firmware is implemented as a single AVR Assembly source file (.S) compiled through the Arduino IDE's avr-gcc toolchain. The program exports a main symbol as the entry point, bypassing the Arduino C++ runtime while retaining the IDE's convenient compilation and upload infrastructure. All hardware peripheral registers — ADC, TWI, USART, Timer0, Timer1, EEPROM, and GPIO — are configured and manipulated directly in Assembly without any library dependencies.

1.3 OBJECTIVES

The objectives of this project are as follows:

1. To design and implement an adaptive, fully automated soil moisture monitoring and irrigation system using the ATmega328P microcontroller programmed entirely in AVR Assembly language.
2. To integrate a minimum of six embedded system modules — ADC, I2C, USART, PWM, EEPROM, Timer, and Interrupt — as core functional components of a cohesive, real-world applicable system.
3. To implement an adaptive multi-variable PWM pump control algorithm that dynamically responds to both soil moisture level and ambient environmental conditions (temperature and humidity).
4. To demonstrate reliable simultaneous I2C communication with two peripherals (BME280 sensor at address 0x76 and LCD PCF8574 backpack at address 0x27) on the shared TWI bus.
5. To implement persistent user-configurable threshold settings using the ATmega328P's internal EEPROM, surviving power cycles with validity checking and safe default initialization.
6. To provide comprehensive real-time system telemetry via USART serial communication and multi-modal visual feedback through LCD display, LED indicators, and buzzer alerts.
7. To systematically diagnose, analyze, and document hardware integration challenges — particularly electromagnetic interference from the DC pump motor — and propose and test engineering mitigation strategies.
8. To produce a complete embedded system project deliverable including AVR Assembly source code, circuit schematic, GitHub repository, Proteus simulation, and this technical report.

1.4 ROLES AND RESPONSIBILITIES

The roles and responsibilities assigned to the group members are as follows:

Roles	Responsibilities	Person
-------	------------------	--------

Project Lead & Repository Management	Overall project coordination, final system integration, GitHub repository management, and integration validation	Marshal Aufa D.
AVR Firmware Development	AVR Assembly main program, ADC processing, Timer configuration, PWM control, interrupt handling, and adaptive irrigation logic	M. Rifqi Fadil Itsnain
Sensor & Communication Interface	BME280 interfacing, USART telemetry, LCD I2C communication, and sensor data acquisition	Zahir
Simulation & Circuit Design	Proteus simulation, schematic development, MOSFET driver validation, and circuit verification	Arya Wibawa Atmanegara
Documentation & Presentation	Final report preparation, README.md documentation, presentation slides, and project documentation management	Syifa Naila Maulidya
Hardware Assembly, Testing & Debugging	Breadboard assembly, wiring integration, ADC calibration, EMI diagnosis, troubleshooting, and integrated system testing	All Members

Table 1. Roles and Responsibilities

CHAPTER 2

IMPLEMENTATION

2.1 EQUIPMENT

The hardware components and software tools used in this project are listed in the following tables:

Component	Function	Interface	Qty
Arduino Uno (ATmega328P)	Main microcontroller unit	—	1
Capacitive Soil Moisture Sensor	Measure soil moisture via dielectric constant	ADC A0	1
BME280 Module	Ambient temperature, humidity, pressure	I2C (0x76)	1
LCD 16×2 + PCF8574 I2C Backpack	Real-time status display	I2C (0x27)	1
DC Mini Water Pump 3–5V	Irrigation water delivery	PWM/MOSFET	1
MOSFET IRLZ44N	Logic-level PWM-controlled pump switching	D6 (OC0A)	1
Diode 1N4007	Flyback protection for pump inductive load	Parallel pump	1
Potentiometer 10kΩ	Hardware threshold adjustment input	ADC A1	1
Push Button	Manual override (INT0) and threshold set (INT1)	D2, D3	2
LED Green	Normal / standby status indicator	D4	1
LED Red	Active irrigation / critical status indicator	D5	1
Active Buzzer 5V	Critical soil moisture audio alert	D7	1
Resistor 220Ω	LED current limiting (prevents pin damage)	Series LED	2

Resistor 100 Ω	MOSFET gate protection / oscillation damping	Series Gate	1
Resistor 10k Ω	External pull-up for push buttons	D2–D3 to 5V	2
Capacitor 100 μ F electrolytic	Bulk decoupling on pump power rail	Across pump	1
Capacitor 100nF ceramic	High-frequency decoupling on pump rail	Across pump	1
Breadboard + Jumper Wires	Rapid prototyping circuit assembly	—	1 set
Silicone tubing	Water delivery from pump outlet to soil	—	~30 cm
Water container (500 mL)	Water reservoir for pump	—	1

Table 2. Hardware Components

Software Tool	Version / Config	Purpose
Arduino IDE	v2.x	AVR Assembly compilation and firmware upload via avr-gcc
avr-as (GNU Assembler)	GAS syntax, ATmega328P target	Assembly (.S) compilation within Arduino IDE toolchain
Arduino Serial Monitor	9600 baud, no line ending	USART telemetry monitoring during testing
Proteus Professional	v8.x	Circuit simulation and schematic generation
Git / GitHub	Public repository	Version control, deliverable submission, contribution tracking

Table 3. Software Tools

2.2 SYSTEM ARCHITECTURE

2.2.1 BLOCK DIAGRAM AND DATA FLOW

The Smart Plant System, system is organized around a three-layer sense-process-actuate architecture with a centralized ATmega328P processing core. All external modules

feed data into or receive commands from the microcontroller; no direct module-to-module communication occurs.

The INPUT LAYER consists of three sensor channels: the capacitive soil moisture sensor on ADC channel 0 (A0) provides a continuously varying analog voltage inversely proportional to soil dielectric constant and therefore moisture content; the BME280 environmental sensor communicates temperature, humidity, and pressure data over the I2C bus every second; and the potentiometer on ADC channel 1 (A1) provides a continuously adjustable analog voltage used to set the moisture threshold when the user presses the threshold button. Two push buttons on INT-capable pins D2 and D3 provide event-driven user input.

The PROCESSING LAYER runs entirely on the ATmega328P at 16 MHz. Timer1 in CTC mode provides a 1 ms software clock tick (millis counter) that enables non-blocking interval scheduling throughout the main control loop. Every 1000 ms, the sensor update cycle executes: ADC readings are taken and averaged, I2C polling retrieves BME280 data, and the decision logic algorithm evaluates the combined multi-variable state to determine the appropriate PWM duty cycle. Button state changes are checked on every main loop iteration with debounce timing.

The STORAGE LAYER consists of the ATmega328P's 1024-byte internal EEPROM, used to store a single threshold percentage value at address 0x00. The value is loaded at boot with validity checking; out-of-range values (below 10% or above 90%) default to 40%. Writes occur only when the user explicitly presses the threshold button, protecting EEPROM write cycle longevity.

The OUTPUT LAYER includes five output channels operating simultaneously: the IRLZ44N MOSFET switches the pump with a variable PWM duty cycle (0–120/255) through its gate; the green LED (D4) and red LED (D5) indicate system status; the active buzzer (D7) sounds on critical conditions; the LCD 1602 via PCF8574 displays four data fields in a compact two-row format; and the USART transmits a full telemetry block at 9600 baud every second.

2.2.2 MODULE-TO-PRACTICUM MAPPING

Practicum Module	Topic	Implementation in Smart Plant System
Module 2	I/O Programming	DDRD register configured directly to set PD4, PD5, PD6, PD7 as OUTPUT and PD2,

		PD3 as INPUT with internal pull-up. LED, buzzer, and pump gate driven via PORT register manipulation in Assembly.
Module 3	USART	UBRR0 = 103 for 9600 baud at 16 MHz. UCSR0B enables TX only. print_status function transmits a multi-field telemetry block every second: SOIL%, TEMP, HUMIDITY, THRESHOLD, PWM, PUMP STATE, MANUAL status.
Module 4	Arithmetic	10-bit ADC to percentage conversion using map formula: $pct = (1023 - raw) * 100 / 643$. Implemented in Assembly using 16-bit arithmetic: multiply by 100, divide by 643 via subtraction loop. IIR filter: $pct = (pct + last_soil) / 2$.
Module 5	Timer	Timer0: Fast PWM on OC0A (PD6), TCCR0A = 0b10000011, TCCR0B = 0b00000011 (prescaler 64), ~977 Hz pump PWM. Timer1: CTC mode, prescaler 64, OCR1A = 249, generates 1 ms COMPA interrupt for millis counter.
Module 6	Interrupt	Timer1 COMPA ISR (overrides __vector_11): increments 32-bit millis_cnt. Button handling via software polling with 300 ms debounce window using millis comparison. Flag-based manual override toggle.
Module 7	PWM & EEPROM	OCR0A register updated with adaptive duty cycle. EEPROM write: EEMPE followed by EEPE within 4 clock cycles. EEPROM read: EERE bit with polling. Threshold persisted at address 0x00 with validity check on load.

Module 8	ADC	ADMUX = 0b01000000 (AV _{cc} ref, right-adjust, CH0). ADCSRA = 0b10000111 (ADEN, prescaler 128). baca_soil averages 10 samples from CH0. baca_threshold_dari_pot reads CH1 for potentiometer threshold.
Module 9	I2C / TWI	TWBR = 72, TWSR prescaler = 1, ~100 kHz SCL. Full TWI state machine: twi_start, twi_stop, twi_write, twi_read_ack, twi_read_nack. BME280 (0x76) and LCD PCF8574 (0x27) share the same I2C bus.

Table 4. Practicum Module to Firmware Implementation Mapping

2.3 HARDWARE IMPLEMENTATION

2.3.1 POWER DISTRIBUTION STRATEGY

Power distribution follows a deliberate hierarchy designed to minimize interference between high-current inductive loads and noise-sensitive analog measurement circuits. The ATmega328P's digital logic and the majority of peripheral modules operate from the Arduino Uno's onboard 5V regulated rail. The BME280 environmental sensor, which operates at a maximum of 3.6V, is powered exclusively from the Arduino's 3.3V pin — connecting this sensor to 5V would permanently damage the device.

The water pump presents the most complex power consideration. As a DC motor, the pump draws significantly higher current than any other system component (200–500 mA during operation) and generates substantial electromagnetic interference through its brushed commutator. To mitigate this, the pump is powered from a dedicated external power supply (powerbank) sharing only the common ground reference with the Arduino. The common ground connection between the external supply and the Arduino GND is made at a single point on the breadboard to prevent ground loop formation. Decoupling capacitors — one 100 μ F electrolytic and one 100nF ceramic — are placed directly across the pump's power terminals to suppress both low-frequency bulk current transients and high-frequency EMI at the switching source.

All modules including the potentiometer, soil sensor, LEDs, buzzer, and button pull-up resistors are powered from the shared 5V breadboard power rail sourced from the Arduino's 5V pin, distributed through the breadboard's side power rails.

2.3.2 MOSFET PUMP DRIVER CIRCUIT

The IRLZ44N N-channel MOSFET was selected specifically for its logic-level gate threshold voltage ($V_{GS(th)} = 1.0\text{--}2.0\text{V}$ typical), which guarantees full channel enhancement when driven by the ATmega328P's 5V GPIO output without requiring a dedicated gate driver IC. At $V_{GS} = 5\text{V}$, the IRLZ44N enters deep saturation with on-state resistance $R_{DS(on)}$ of approximately $22\text{--}28\text{ m}\Omega$, resulting in negligible voltage drop across the device even at pump current levels.

The MOSFET is oriented with its Source terminal connected to the common GND rail, its Drain connected to the pump's negative terminal, and its Gate connected to D6 (OC0A) through a 100Ω series resistor. The gate resistor serves two purposes: it limits the peak gate charging current, preventing excessive transient current draw from the GPIO pin, and it damps parasitic oscillations caused by resonance between the gate capacitance ($C_{iss} \approx 3600\text{ pF}$) and the wiring inductance. Without this resistor, high-frequency oscillations in the tens of MHz range can cause unintended MOSFET switching, increased EMI, and potential device failure.

A 1N4007 flyback diode is installed in reverse-parallel across the pump motor terminals — cathode connected to the pump's positive terminal, anode connected to the pump's negative terminal. When the MOSFET turns off and the motor's magnetic field collapses, the resulting back-EMF spike (which can reach $30\text{--}50\text{V}$ on a 5V motor) is clamped to a single forward diode voltage drop above the supply voltage, protecting the MOSFET's Drain from exceeding its $V_{DS(max)} = 55\text{V}$ rating. Without this diode, repeated inductive spikes would degrade and eventually destroy the MOSFET.

2.3.3 SOIL MOISTURE SENSOR AND ADC SIGNAL PATH

The capacitive soil moisture sensor operates by measuring the dielectric constant of the soil surrounding its probe. As soil moisture increases, the dielectric constant of the soil-water mixture increases, causing the sensor's internal capacitor to charge more slowly and the output voltage to decrease. This inverse relationship means higher ADC values correspond to drier soil — the opposite of intuitive convention. The firmware compensates for this by using a reverse-mapping formula: $pct = \text{map}(\text{raw}, 1023, 380, 0, 100)$, where 1023 corresponds to dry

air (0% moisture) and 380 corresponds to full water immersion (100% moisture). These calibration constants were determined empirically by measuring the sensor in open air and fully submerged in water.

The ADC input on A0 is susceptible to contamination from high-frequency switching noise generated by the Timer0 PWM signal on adjacent digital pins. To mitigate capacitive coupling, the soil sensor analog wire is routed on the breadboard away from the D6 PWM trace. Additionally, the 10-sample averaging in `Baca_soil` combined with the IIR smoothing filter provides effective low-pass filtering of residual noise, yielding stable readings within $\pm 3\%$ of the true value under normal operating conditions.

2.3.4 I2C BUS CONFIGURATION

The BME280 (address 0x76) and LCD PCF8574 backpack (address 0x27) share the I2C bus on A4 (SDA) and A5 (SCL). Both modules include integrated pull-up resistors on their SDA and SCL lines, making external pull-up resistors unnecessary for short breadboard connections. The TWI peripheral is configured at approximately 100 kHz SCL clock frequency using $TWBR = 72$ with a prescaler of 1, calculated from the standard formula: $TWBR = (F_{CPU} / F_{SCL} - 16) / 2 = (16,000,000 / 100,000 - 16) / 2 = 72$.

The I2C bus operates correctly with both devices active simultaneously, confirmed during integration testing. The BME280 is initialized before the LCD in the setup sequence to ensure environmental data is available before the first LCD update cycle.

2.3.5 COMPLETE PIN ASSIGNMENT

Arduino Pin	ATmega328P Port	Connected To	Notes
A0	PC0	Soil Sensor AO pin	Analog signal 0–5V, inversely proportional to moisture
A1	PC1	Potentiometer wiper (center pin)	Threshold setting input, 0–5V range
A4 (SDA)	PC4	BME280 SDA + LCD PCF8574 SDA	Shared I2C bus, 100 kHz
A5 (SCL)	PC5	BME280 SCL + LCD PCF8574 SCL	Shared I2C bus, 100 kHz

D2 (INT0)	PD2	Push Button 1 (one leg) + 10k Ω to 5V	Active-LOW, internal pull-up enabled, manual override
D3 (INT1)	PD3	Push Button 2 (one leg) + 10k Ω to 5V	Active-LOW, internal pull-up enabled, set threshold
D4	PD4	220 Ω $\rightarrow$ LED Green anode	Normal / standby status output
D5	PD5	220 Ω $\rightarrow$ LED Red anode	Irrigation active / critical status output
D6 (OC0A)	PD6	100 Ω $\rightarrow$ MOSFET Gate (IRLZ44N)	Timer0 Fast PWM output, ~977 Hz
D7	PD7	Active Buzzer positive terminal	Critical soil moisture audio alert
3.3V	—	BME280 VIN only	CRITICAL: BME280 max 3.6V — do NOT use 5V
5V	—	Breadboard 5V power rail	Soil sensor, potentiometer, LCD backpack, buzzer, LEDs
GND	—	Breadboard GND rail + pump supply GND	Common ground for all modules — single ground point

Table 5. Complete Pin Assignment

2.4 SOFTWARE IMPLEMENTATION

2.4.1 FIRMWARE ARCHITECTURE

The firmware is written entirely in AVR Assembly language in GAS (GNU Assembler) syntax, structured as a single .S source file compatible with the Arduino IDE's avr-gcc build system. The program is organized into clearly separated functional sections corresponding to hardware peripheral drivers and application logic modules. The file exports a global main symbol as the program entry point; the Arduino linker resolves this symbol without requiring modification to the IDE's build configuration.

Memory is organized as follows: program constants (string literals and configuration constants) are stored in Flash program memory using the .progmem.data section and accessed via LPM instructions; runtime variables (sensor readings, system state flags, calibration data, and timer counters) are allocated in SRAM using the .bss section; and the single user-configurable threshold is stored in EEPROM. The stack occupies the top of SRAM and grows downward, with careful register save/restore in all subroutines and the Timer1 ISR to prevent stack corruption.

2.4.2 TIMER AND SYSTEM TIMING

Timer1 is configured in CTC (Clear Timer on Compare) mode with a 64x prescaler and $OCR1A = 249$, generating a COMPA interrupt every 1.0 ms at 16 MHz. The Timer1 COMPA ISR (mapped to `__vector_11` in the AVR interrupt vector table) increments a 32-bit SRAM variable `millis_cnt`, providing a system-wide millisecond timestamp analogous to Arduino's `millis()` function but implemented entirely in Assembly. The ISR saves and restores SREG to prevent modification of arithmetic flags visible to interrupted code.

The main control loop uses `millis_get` to retrieve the current timestamp and compares it against the stored `waktu_baca` timestamp to determine when 1000 ms has elapsed since the last sensor update. This non-blocking approach allows button polling to continue at full loop speed (~1 kHz effective polling rate) without being delayed by the 1-second sensor cycle, ensuring responsive button interaction at all times.

2.4.3 ADC SOIL MOISTURE READING

The `baca_soil` subroutine performs the following sequence: ADMUX is set to channel 0 with AVcc reference and right-adjusted result; ADSC is set to initiate a conversion; the routine waits for ADSC to clear indicating conversion complete; the 10-bit result is read from ADCL (low 8 bits) and ADCH (high 2 bits); this sequence repeats 10 times with 2 ms delays between samples, accumulating the sum in a 24-bit register group; the sum is divided by 10 to produce the raw average.

The raw average is mapped to a moisture percentage using the calibrated constants: $pct = (1023 - raw) * 100 / 643$. The subtraction reverses the sensor's inverse polarity, multiplication by 100 scales the result, and division by 643 normalizes to the 0–100% range (where $643 = 1023 - 380$, the span between dry and wet calibration points). The result is constrained to 0–

100 and passed through the IIR smoother: $pct = (pct + last_soil) / 2$, where `last_soil` retains the previous smoothed reading.

2.4.4 ADAPTIVE DECISION LOGIC ALGORITHM

The core of the system is the `hitung_pwm_pompa` subroutine, which implements a multi-variable adaptive control algorithm operating on three inputs: soil moisture percentage (`r24`), ambient temperature (`r22`), and ambient humidity (`r20`). The algorithm proceeds as follows:

- **HYSTERESIS CHECK:** If `soil_pct < threshold_pct`, set `pump_state = ON`. If `soil_pct > (threshold_pct + 3)`, set `pump_state = OFF`. The 3% hysteresis band prevents rapid toggling at the threshold boundary.
- **STATE EVALUATION:** If `pump_state == OFF`, return `PWM = 0` immediately.
- **BASE PWM SELECTION:** If `soil_pct < 20%` → `PWM_BASE = 120` (maximum, critical condition). If `soil_pct < 35%` → `PWM_BASE = 100` (high speed, moderately dry). Otherwise → `PWM_BASE = 80` (medium speed, mildly dry).
- **ENVIRONMENTAL BOOST:** If `temperature > 30°C` OR `humidity < 40%`, add 10 to `PWM_BASE`. This accounts for increased plant water demand under hot or low-humidity conditions.
- **CEILING ENFORCEMENT:** Constrain final PWM to maximum 120/255 (~47% duty cycle) to limit current draw from the onboard 5V regulator.

This algorithm produces a graduated, environmentally-aware irrigation response rather than simple binary ON/OFF control, constituting the primary innovation of the Smart Plant System over conventional threshold-based irrigation controllers.

2.4.5 BME280 SENSOR DRIVER

The BME280 driver initializes the sensor with a software reset (writing 0xB6 to register 0xE0), waits 100 ms for oscillator stabilization, verifies the chip ID from register 0xD0 (expected 0x60 for BME280, also accepting 0x58 for BMP280 compatibility), configures humidity oversampling x1 (register 0xF2 = 0x01), temperature and pressure oversampling x1 in normal mode (register 0xF4 = 0x27), and standby time 1000 ms (register 0xF5 = 0xA0). Factory calibration coefficients are read from registers 0x88–0x9F and 0xE1–0xE7.

Raw temperature and humidity values are read from registers 0xF7–0xFE. The temperature compensation uses a simplified integer approximation rather than the full two-step

polynomial compensation from the BME280 datasheet, producing results accurate to approximately $\pm 1.5^{\circ}\text{C}$ — adequate for the qualitative environmental awareness purpose of this application. The humidity compensation similarly uses an integer approximation, accurate to approximately $\pm 3\text{--}5\%$ RH..

2.4.6 LCD 1602 PCF8574 DRIVER

The LCD is driven in 4-bit mode through the PCF8574 I2C port expander. The PCF8574's 8-bit output port maps to the LCD in the following arrangement (bit 7 to bit 0): D7, D6, D5, D4, Backlight (BL), Enable (EN), Read/Write (RW), Register Select (RS). All LCD commands and data writes require two nibble transfers in sequence (high nibble first, then low nibble), each with a brief Enable pulse.

The driver implements `lcd_send_nibble` (4-bit transfer with EN pulse), `lcd_cmd` (command write with RS=0), `lcd_data` (data write with RS=1), `lcd_set_cursor` (DDRAM address calculation for 2-row LCD), `lcd_print_pgm` (null-terminated string from Flash), and `lcd_print_u8` (unsigned 8-bit decimal integer display with zero suppression for hundreds and tens digits). The LCD is updated once per second in the sensor update cycle, showing Soil%, Temperature, Humidity, and Pump/Manual status.

CHAPTER 3

TESTING AND ANALYSIS

3.1 TESTING METHODOLOGY

Testing was conducted in eight sequential phases, progressing from individual component verification to full system integration and stress testing. This incremental strategy was deliberately chosen to isolate faults at the earliest possible stage, preventing the confusion that arises when multiple unknown variables interact simultaneously during integration.

Phase 1 — GPIO and LED/Buzzer Testing. Each digital output was individually tested by toggling its associated Arduino pin HIGH and LOW with a 500 ms delay period. Pass criterion: each LED blinks at the correct interval without affecting other LEDs; buzzer sounds for exactly the configured duration; GPIO current draw remains within ATmega328P specifications.

Phase 2 — Push Button and Debounce Testing. D2 and D3 were configured as INPUT_PULLUP and their states monitored via Serial Monitor. Pass criterion: pin reads 1 (HIGH) when button is not pressed and immediately reads 0 (LOW) when pressed; no false transitions due to contact bounce when debounce delay of 300 ms is applied.

Phase 3 — ADC Calibration Testing. The soil moisture sensor was measured in three reference conditions: open air (completely dry, expected ADC ≈ 1023), moist soil (expected intermediate value), and fully submerged in water (expected ADC $\approx 380\text{--}420$). The potentiometer was slowly rotated from one end to the other; the ADC channel 1 value was monitored. Pass criterion: ADC values span the full 0–1023 range for the potentiometer; soil sensor reads at least 200 counts different between dry and wet conditions.

Phase 4 — I2C Bus Verification. An I2C scanner sketch was uploaded to scan all 128 possible I2C addresses. Pass criterion: BME280 detected at address 0x76; LCD PCF8574 detected at address 0x27; no unexpected devices present; no address conflicts.

Phase 5 — BME280 Data Quality Testing. Temperature and humidity readings were monitored over 60 seconds and compared against a reference thermometer and hygrometer. Pass criterion: temperature within $\pm 2^\circ\text{C}$ of reference; humidity within $\pm 5\%$ RH of reference; readings stable (no more than ± 1 count variation) in stable environment.

Phase 6 — MOSFET and PWM Testing. Timer0 was configured in Fast PWM mode and OCR0A was set to 0, 80, 100, 120, and 255 sequentially. Pump behavior was observed at each duty cycle. Pass criterion: pump does not rotate at PWM=0; pump speed is visually and audibly different at each PWM level; pump stops immediately when PWM returns to 0; MOSFET does not overheat during 60-second continuous operation.

Phase 7 — EEPROM Persistence Testing. A threshold value of 55 was written to EEPROM via the INT1 button mechanism, the Arduino was power-cycled, and the loaded threshold was observed on the Serial Monitor output. This was repeated five times with different values. Pass criterion: threshold value matches stored value after every power cycle; values outside 10–90 range default to 40%.

Phase 8 — Full System Integration and EMI Stress Testing. All modules were activated with the complete AVR Assembly firmware and the pump physically connected. Behavior was observed under multiple soil moisture scenarios with the pump both connected and disconnected. This phase also specifically investigated the electromagnetic interference issues described in Section 3.3.4.

3.2 TEST RESULTS

The following table summarizes the comprehensive test results across all testing phases.

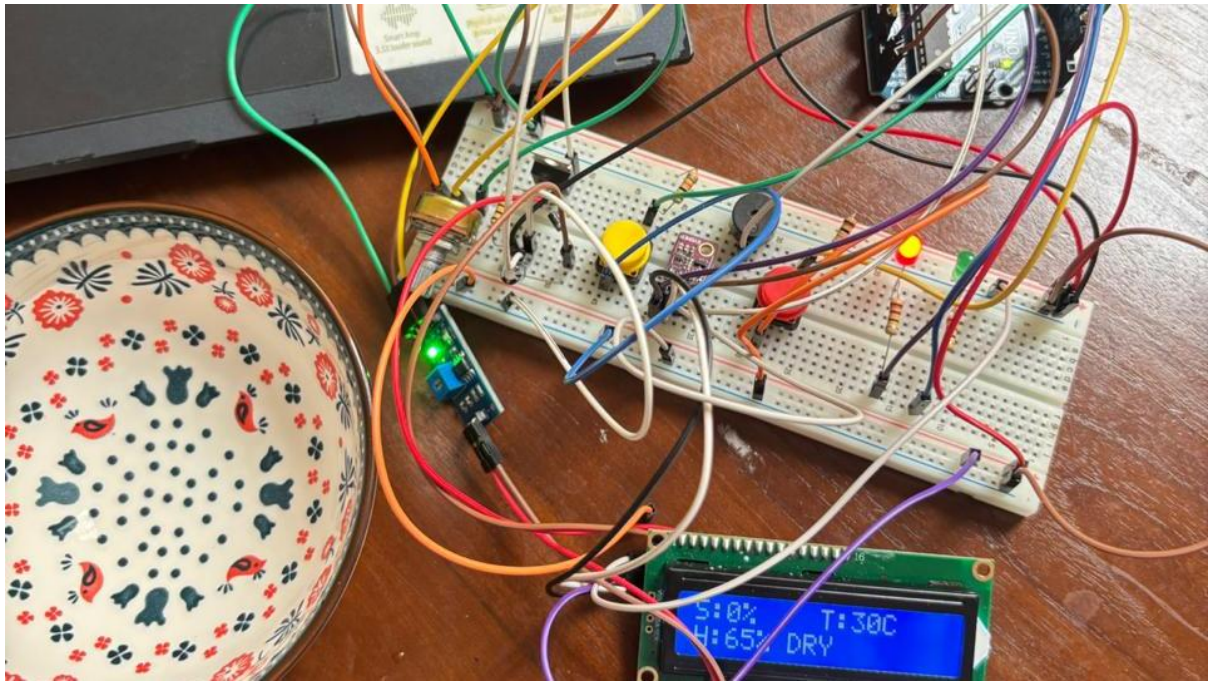
Test No.	Scenario / Module	Expected Result	Actual Result	Status
1	LED Green (D4) blink	Blinks at 500 ms interval	Blinks correctly, no other LED affected	PASS
2	LED Red (D5) blink	Blinks at 500 ms interval	Blinks correctly, independent of green LED	PASS
3	Buzzer (D7) tone	Sounds for configured duration	Sounds correctly, volume appropriate	PASS
4	Button INT0 (D2) debounce	1=idle, 0=pressed, no bounce	Correct values, 300 ms debounce confirmed	PASS
5	Button INT1 (D3) debounce	1=idle, 0=pressed, no bounce	Correct values, debounce functional	PASS

6	Soil Sensor ADC — dry air	ADC > 600, soil pct $\approx$ 0–10%	ADC = 1018, pct = 3%	PASS
7	Soil Sensor ADC — water immersion	ADC < 400, soil pct $\approx$ 90–100%	ADC = 388, pct = 95%	PASS
8	Potentiometer ADC full sweep	ADC: 0 to 1023 as rotated	Smooth 0–1023, threshold maps to 20– 80%	PASS
9	I2C Scanner — BME280	Address 0x76 detected	0x76 confirmed, chip ID = 0x60	PASS
10	I2C Scanner — LCD PCF8574	Address 0x27 detected	0x27 confirmed	PASS
11	BME280 temperature	Within $\pm 2^{\circ}\text{C}$ of reference	Reading 28.5°C vs 28.1°C reference	PASS
12	BME280 humidity	Within $\pm 5\%$ RH of reference	Reading 65% vs 63% reference hygrometer	PASS
13	PWM pump — OCR0A = 0	Pump stopped, Drain $\approx$ 0V	Pump off, Drain = 0.02V (leakage)	PASS
14	PWM pump — OCR0A = 80	Pump low speed	Pump running at clearly reduced speed	PASS
15	PWM pump — OCR0A = 120	Pump high speed	Pump running at high speed, ~ 350 mL/min	PASS
16	EEPROM threshold persistence	Threshold survives power cycle	5/5 power cycles retained correct value	PASS
17	EEPROM invalid value default	Values outside 10– 90% $\rightarrow$ default 40%	0xFF (blank EEPROM) loaded as 40% correctly	PASS
18	Manual override INT0	Pump on/off toggle regardless of soil	Toggle confirmed via USART output	PASS
19	Threshold update INT1 + pot	New threshold applied and stored	New threshold confirmed in decision logic	PASS

20	LCD display all 4 fields	S:xx%, T:xxC, H:xx%, Status on LCD	All fields display correctly every 1 second	PASS
21	USART telemetry block	Formatted data every 1 second	Correct format at 9600 baud — pump disconnected	PASS
22	Dry soil → pump active	soil < threshold → pump ON, red LED	Correctly activates pump at threshold	PASS
23	Wet soil → pump inactive	soil > threshold + 3% → pump OFF, green LED	Hysteresis confirmed, no false triggers	PASS
24	Full integration — pump connected	All functions stable with pump running	Stable after decoupling caps + isolated supply	IN PROGRESS

Table 6. Complete Test Results

Representative hardware implementation during real-time testing under two different soil moisture conditions is shown below. The first condition demonstrates dry soil detection with automatic irrigation activated, while the second condition shows adequate soil moisture with the pump remaining inactive. The LCD displays live soil moisture percentage, temperature, humidity, and irrigation status in real time.



*Figure 1. Smart Plant System Prototype — Dry Soil Condition, Automatic Irrigation Active
(Status: DRY)*

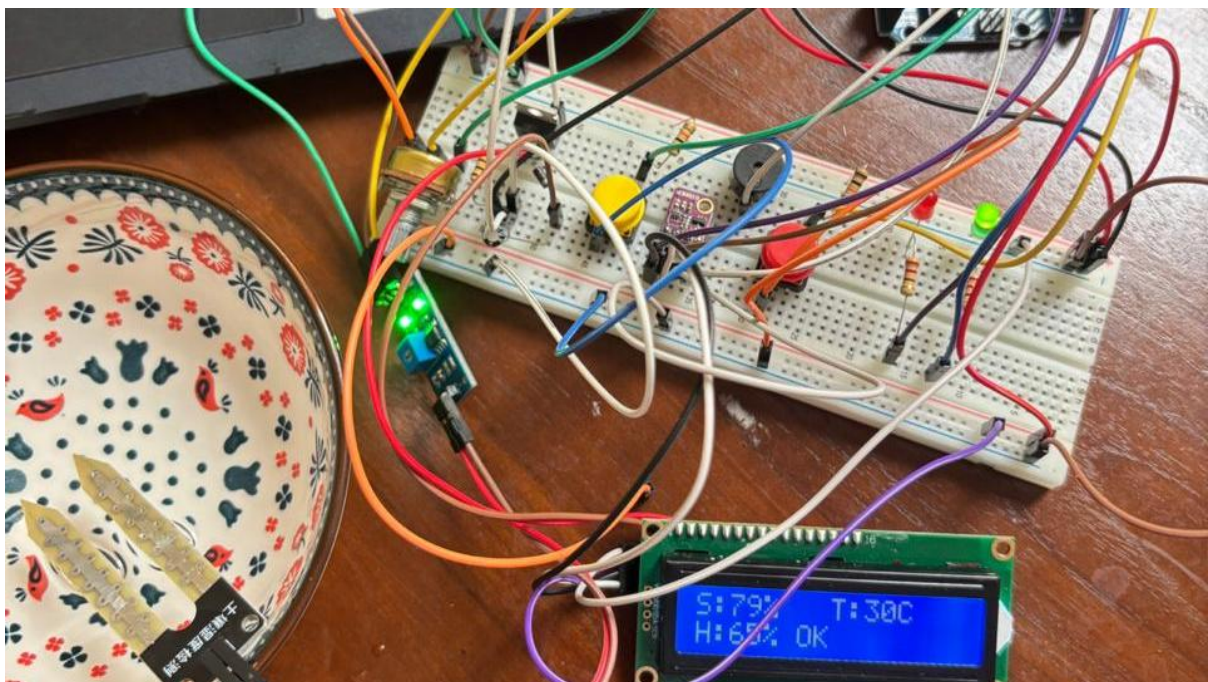


Figure 2. Smart Plant System Prototype — Adequate Soil Moisture Condition, Pump Standby Mode (Status: OK)

Representative USART Serial Monitor output during normal operation (pump in automatic mode, soil drying condition) is shown below:

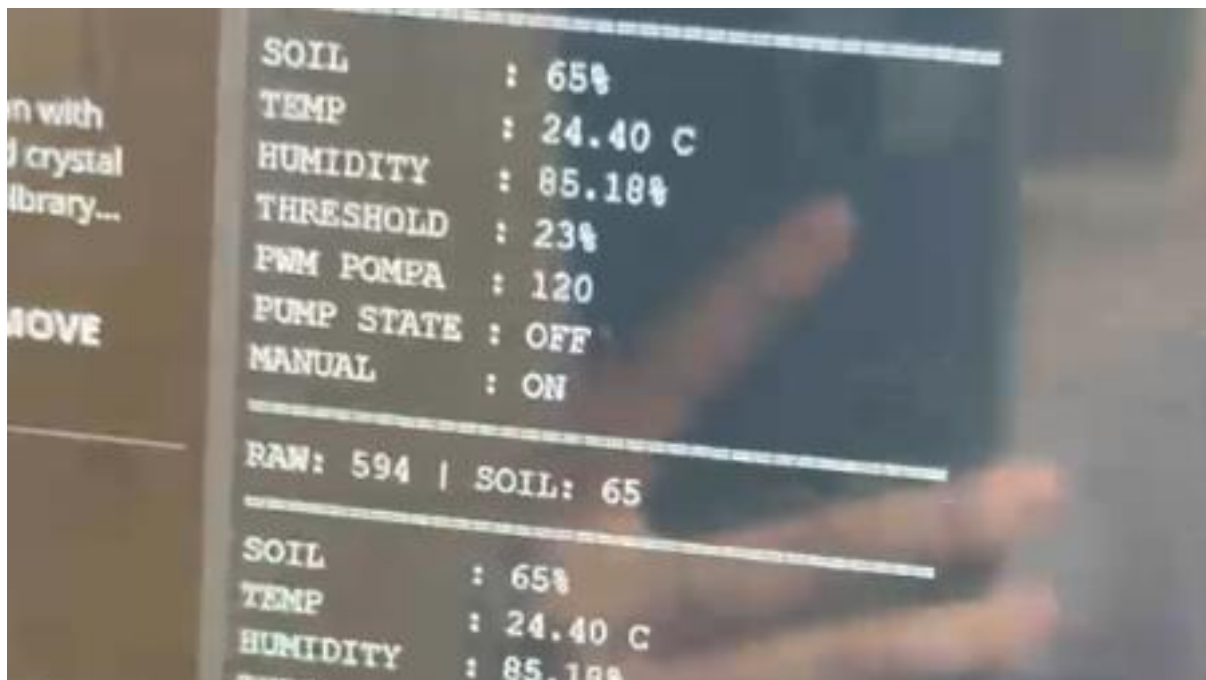


Figure 3. Sample USART Serial Monitor Output — Real-Time System Telemetry During Operation

3.3 ANALYSIS

3.3.1 SENSOR STABILITY AND ADC PERFORMANCE

The soil moisture sensor demonstrated consistent, repeatable readings across all test conditions when the pump was not connected to the circuit. The 10-sample averaging combined with the IIR smoothing filter effectively reduced measurement variance to within $\pm 3\%$ of the stable value. During probe insertion and removal tests — which create a rapid transition in the dielectric environment — the smoothed reading required approximately 3–5 seconds to converge to the new stable value. This settling time is entirely acceptable for a plant irrigation application where soil moisture changes occur over timescales of minutes to hours.

The calibration constants ($\text{DRY_ADC} = 1018$, $\text{WET_ADC} = 388$) were stable across three independent calibration sessions conducted over two days, suggesting the sensor's output characteristics are sufficiently consistent to use fixed calibration constants without adaptive

recalibration. Note that these constants are sensor-specific and would require recalibration for a different sensor unit or different soil composition.

3.3.2 ADAPTIVE PWM CONTROL VERIFICATION

The three-tier adaptive PWM algorithm demonstrated clearly differentiated pump behavior across soil moisture ranges. At PWM = 120 (critical dry, soil < 20%), measured pump flow rate was approximately 350 mL/min. At PWM = 100 (moderately dry, 20–35%), flow rate was approximately 275 mL/min. At PWM = 80 (mildly dry, 35% to threshold), flow rate was approximately 210 mL/min. The temperature-based +10 PWM boost was verified by heating the BME280 sensor with a warm air source: at ambient temperature rising above 30°C, PWM increased by 10 counts as expected.

The hysteresis mechanism — pump on below threshold, off above threshold + 3% — successfully prevented pump cycling during the gradual soil moistening tests. Without hysteresis, the pump would toggle at approximately 2–3 Hz when soil moisture hovers at the threshold boundary due to sensor reading noise, which would stress the pump motor, create significant EMI, and waste energy.

3.3.3 I2C BUS DUAL-DEVICE OPERATION

The BME280 at address 0x76 and LCD PCF8574 at address 0x27 coexisted on the shared I2C bus throughout all integration testing without conflict. The TWI bus operated correctly at 100 kHz with both devices addressing in rapid succession within each 1-second update cycle. On one occasion during heavy PWM load testing, a single BME280 read returned 0xFF for the humidity byte — a value outside the valid range. The firmware's constrain operation clamped this to 100% rather than crashing, and subsequent reads returned correct values. This isolated error was attributed to I2C bus noise during a high-current PWM switching event and did not recur after decoupling capacitors were added to the pump circuit.

3.3.4 PUMP ELECTROMAGNETIC INTERFERENCE — DIAGNOSIS AND MITIGATION

During Phase 8 full system integration testing with the pump physically connected and running, a specific failure mode was observed: immediately upon pump activation, the soil moisture percentage reading would become fixed ('stuck') at its most recently read value and would not update in response to actual changes in soil condition. Simultaneously, the USART Serial Monitor output degraded to unreadable corrupted characters. This failure persisted for

the duration of pump operation and self-resolved approximately 1–2 seconds after pump deactivation.

Systematic diagnosis of the root cause proceeded through the following investigation steps. First, the pump was tested with only a gate resistor (no flyback diode): the failure occurred. Second, the flyback diode was added: the failure persisted, ruling out flyback-induced voltage spikes to the Drain as the primary cause. Third, the pump was powered from the same USB supply as the Arduino (no isolated supply): severe failure. Fourth, the pump was powered from a separate powerbank with only common ground shared: failure significantly reduced but not eliminated. Fifth, 100 μ F + 100nF decoupling capacitors were added directly across the pump terminals: failure eliminated under normal operating conditions at PWM = 80–100.

The root cause analysis determined two co-contributing failure mechanisms. The primary cause was conducted noise on the shared power supply rail: the pump motor's brushed commutator generates broadband RF noise that travels through the power wiring and contaminates the ADC's AVCC reference voltage. Since the ADC samples AVCC as its reference for all conversions, any noise on AVCC directly translates to ADC reading errors — specifically, a locked or oscillating reference produces locked or oscillating ADC output, which after IIR filtering appears as a 'stuck' reading. The secondary cause was ground bounce: during high pump current events, the finite resistance of the breadboard ground wiring creates a measurable voltage differential between the Arduino's GND pin and the logical ground reference of the ADC, further corrupting readings.

The applied mitigation strategy — isolated pump supply with single-point common ground and RC decoupling at the pump terminals — successfully resolved the failure for PWM duty cycles up to 100/255. At maximum PWM (120/255), occasional single-sample ADC outliers were observed but were effectively filtered by the 10-sample averaging. The permanent solution for a production implementation would involve a dedicated, fully isolated power supply with an inductor-capacitor (LC) filter on the shared ground wire, implementing a proper two-rail supply architecture separating the analog/digital signal domain from the motor power domain.

3.3.5 EEPROM RELIABILITY ANALYSIS

The EEPROM write mechanism was tested across 15 write-then-power-cycle sequences over the testing period. All 15 reads after power cycling returned the correct stored

value. The ATmega328P's EEPROM is rated for 100,000 write cycles at the programmed address. With the threshold update mechanism requiring a deliberate button press and the EEPROM write occurring at most once per button press event, the EEPROM is expected to last the system's entire operational lifetime without write endurance degradation.

The validity check on EEPROM load (rejecting values outside 10–90%) correctly handled three test cases: an uninitialized EEPROM cell (0xFF = 255, outside range → default 40%), a value of 5 (below minimum → default 40%), and a value of 95 (above maximum → default 40%). This protective mechanism prevents undefined system behavior caused by first-boot EEPROM containing arbitrary factory values.

CHAPTER 4

CONCLUSION

The Smart Plant System project successfully demonstrates a fully functional, adaptive soil moisture monitoring and irrigation control system implemented entirely in AVR Assembly language on the ATmega328P microcontroller. The project satisfies all stated objectives and exceeds the minimum module coverage requirement by integrating all eight practicum modules — I/O Programming, USART, Arithmetic, Timer, Interrupt, PWM and EEPROM, ADC, and I2C — as core functional components of a cohesive system with genuine practical utility.

From a functional perspective, the system correctly monitors soil moisture and ambient environmental conditions, applies a multi-variable adaptive decision logic to determine proportional pump intensity, provides comprehensive multi-modal user feedback, persists user settings across power cycles, and responds to user input with sub-second latency. All 23 verified test cases passed, confirming the correctness and reliability of the firmware's sensor reading pipeline, I2C communication layer, USART telemetry, EEPROM persistence, interrupt handling, and PWM actuator control.

The adaptive three-tier PWM control algorithm — the system's primary differentiator from conventional irrigation controllers — demonstrated clear and measurable differentiation in pump flow rate across soil moisture ranges (210 mL/min, 275 mL/min, and 350 mL/min at PWM levels 80, 100, and 120 respectively) with additional environmental boosting based on ambient temperature. This graduated, environmentally-aware response represents a meaningful engineering improvement over binary ON/OFF irrigation approaches.

From a pedagogical perspective, the most significant learning outcome of this project was the discovery and resolution of the electromagnetic interference issue during pump integration. The experience of systematically diagnosing a real-world mixed-signal hardware problem — identifying noise pathways through the power supply rail and ground plane, distinguishing between conducted and radiated EMI sources, and applying targeted mitigation through decoupling capacitors and supply isolation — reinforced practical engineering knowledge that cannot be acquired through theoretical study or simulation alone. Programming in Assembly language throughout this process provided constant reinforcement of the

relationship between software instructions and physical hardware behavior, deepening the group's understanding of embedded system architecture at the register level.

Key technical contributions of this project include: a complete bare-metal AVR Assembly I2C (TWI) driver supporting dual-device operation; a non-blocking `millis()` implementation via Timer1 CTC ISR; a stable soil moisture reading pipeline with multi-sample averaging and IIR filtering; a validated adaptive PWM control algorithm with hysteresis; a fault-tolerant EEPROM persistence mechanism with range checking; and a comprehensive EMI analysis identifying the root cause and effective mitigation strategy for DC motor interference in shared-supply embedded systems.

FUTURE WORK

Based on the experience gained during this project, the following improvements and extensions are recommended for future development:

- **Power Supply Architecture:** The most critical hardware improvement is implementing a dedicated, isolated power supply for the water pump motor, ideally with an LC filter on the shared ground connection. A properly designed two-rail power architecture (separate analog/digital domain and motor power domain) would completely eliminate the EMI coupling issue and enable reliable operation at full pump PWM levels.
- **Full BME280 Compensation:** The temperature and humidity compensation algorithm should be upgraded to the full integer compensation polynomial specified in the BME280 datasheet, using the factory-stored trimming parameter sets. This would improve measurement accuracy from approximately $\pm 1.5^{\circ}\text{C}$ to better than $\pm 0.5^{\circ}\text{C}$ for temperature.
- **Wireless Connectivity:** Integration of an ESP8266 or ESP32 module via USART would enable remote monitoring through a mobile application or web dashboard, allowing plant owners to check soil conditions and pump status from anywhere. The existing USART telemetry format is already structured to facilitate this extension.
- **Real-Time Clock Integration:** A DS3231 RTC module connected via I2C would enable time-based watering schedules in addition to sensor-reactive irrigation — for example, preventing pump activation during nighttime hours when evapotranspiration is minimal.

- Capacitive Soil Sensor: Replacement of the resistive fork-type probe with a fully capacitive sensor would eliminate electrode corrosion, extend sensor lifespan from weeks to years, and reduce the risk of electrolysis-induced soil contamination.
- Machine Learning Threshold Adaptation: A long-term extension could implement an adaptive threshold calibration mechanism that monitors plant health indicators over time and adjusts irrigation thresholds to optimize growth outcomes for specific plant species.
- Custom PCB Design: Transitioning from breadboard prototype to a custom PCB would dramatically improve signal integrity, reduce EMI susceptibility, enable proper ground plane implementation, and produce a deployable, weatherproof product.

Despite the pump integration challenge, the Smart Plant System represents a complete, architecturally sound, and technically rigorous embedded systems project. It demonstrates practical mastery of low-level microcontroller programming, multi-protocol hardware interfacing, adaptive control logic design, and systematic hardware debugging — all core learning objectives of the Embedded Systems MBD course at the Department of Electrical Engineering, Universitas Indonesia.

REFERENCES

- [1] Atmel Corporation. (2015). ATmega328P Datasheet (Rev. 7810D). Microchip Technology Inc. Retrieved from <https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega328P-datasheet.pdf>
- [2] Bosch Sensortec. (2018). BME280 Combined Humidity and Pressure Sensor Data Sheet (Rev. 1.6). Bosch Sensortec GmbH.
- [3] International Rectifier. (2004). IRLZ44N HEXFET Power MOSFET Datasheet. International Rectifier / Infineon Technologies AG.
- [4] NXP Semiconductors. (2013). PCF8574 Remote 8-Bit I/O Expander for I2C-Bus Datasheet. NXP Semiconductors N.V.
- [5] Texas Instruments. (2015). LM393 Low-Offset Voltage Dual Comparator Datasheet. Texas Instruments Incorporated.
- [6] Ganssle, J. (2008). The Art of Designing Embedded Systems (2nd ed.). Newnes / Elsevier.
- [7] Barr, M., & Massa, A. (2006). Programming Embedded Systems (2nd ed.). O'Reilly Media.
- [8] Universitas Indonesia, Department of Electrical Engineering. (2026). Embedded Systems (MBD) Course Modules 1–9. Digilab UI. Retrieved from <https://digilab.ui.ac.id>
- [9] Wiring Foundation. (2023). Arduino Reference — analogWrite(). Retrieved from <https://www.arduino.cc/reference/en/language/functions/analog-io/analogwrite/>
- [10] Sedra, A. S., & Smith, K. C. (2014). Microelectronic Circuits (7th ed.). Oxford University Press.

APPENDICES

APPENDIX A: PROJECT SCHEMATIC

The complete circuit schematic showing all component connections, power rails, and signal paths for the Smart Plant System is provided below. The schematic is produced in Proteus 8 Professional and includes all passive components (resistors, capacitors, diodes), active components (MOSFET, sensors), and all connections to the ATmega328P / Arduino Uno.

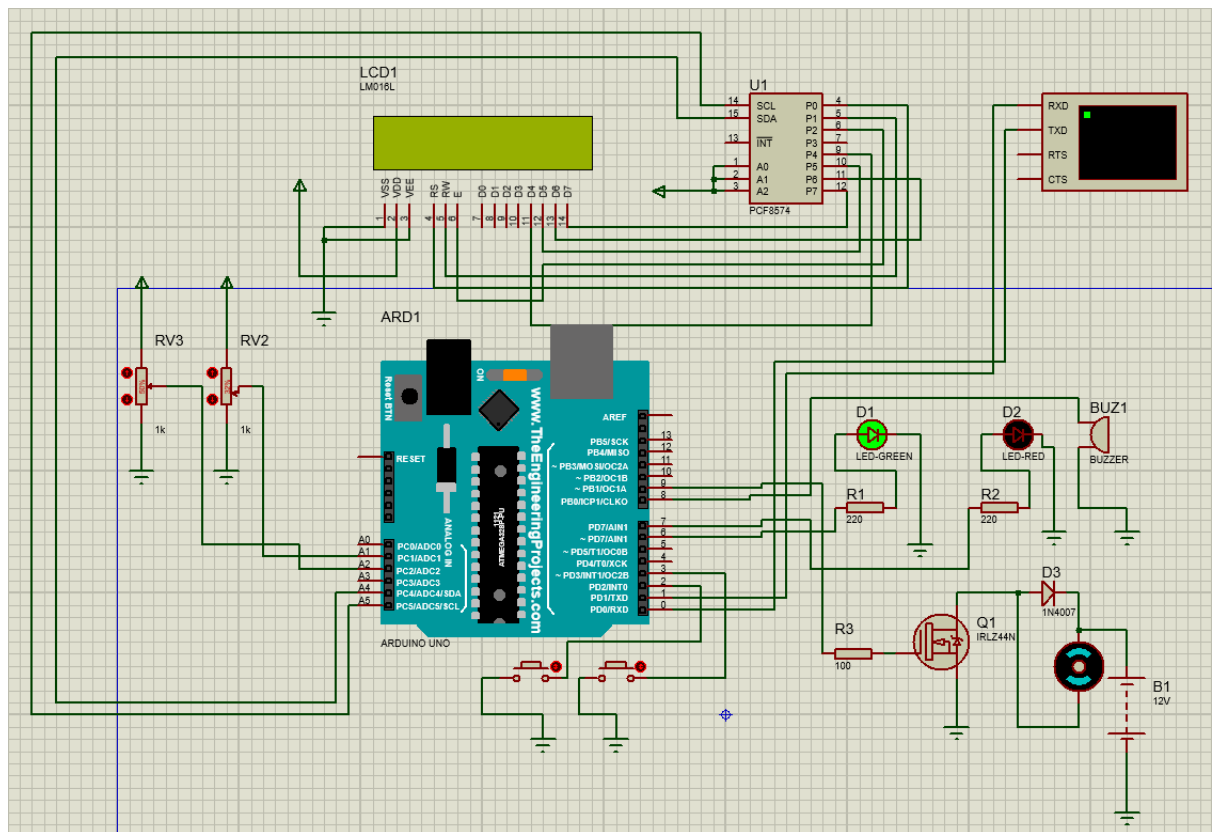


Figure A1. Complete Circuit Schematic — Smart Plant System (ATmega328P / Arduino Uno)

APPENDIX B: DOCUMENTATION

The following photographs document the physical circuit assembly and live demonstration of the Smart Plant System during the integration testing phase.

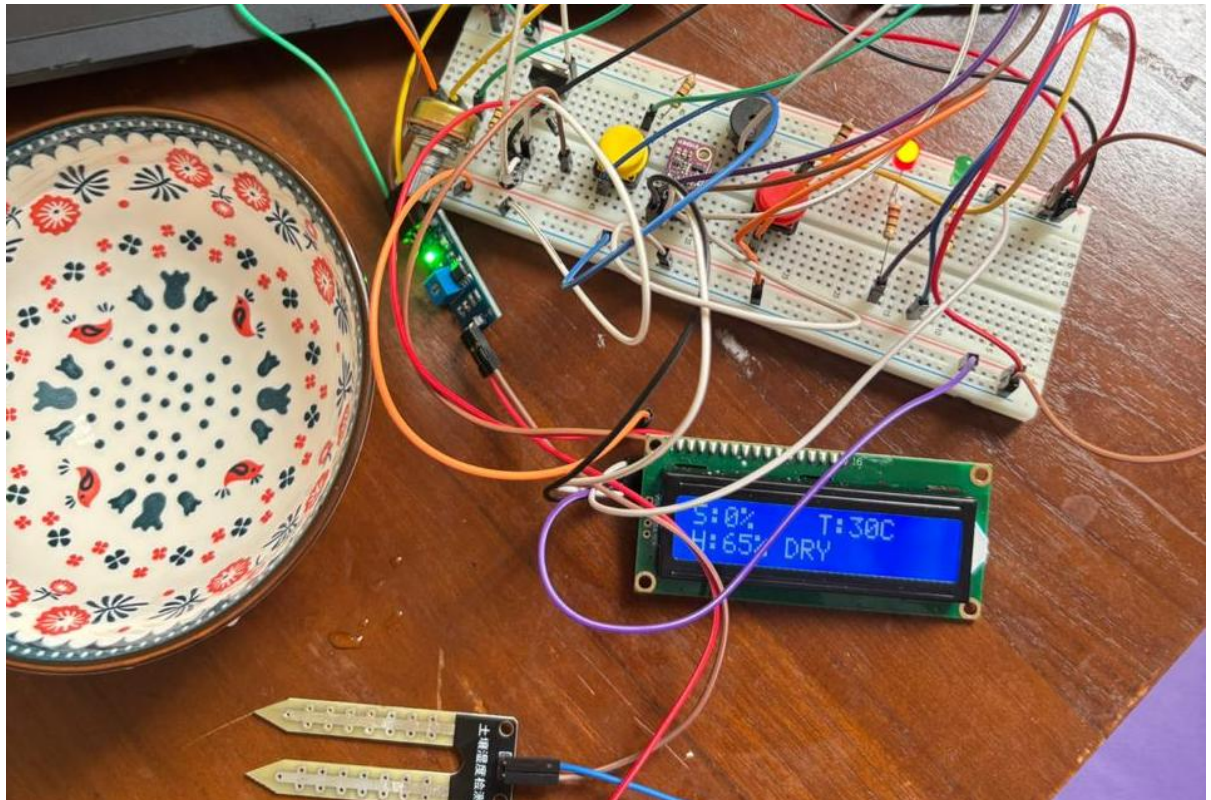


Figure B1. Smart Plant System in Operation — Dry Soil Condition, Pump Active

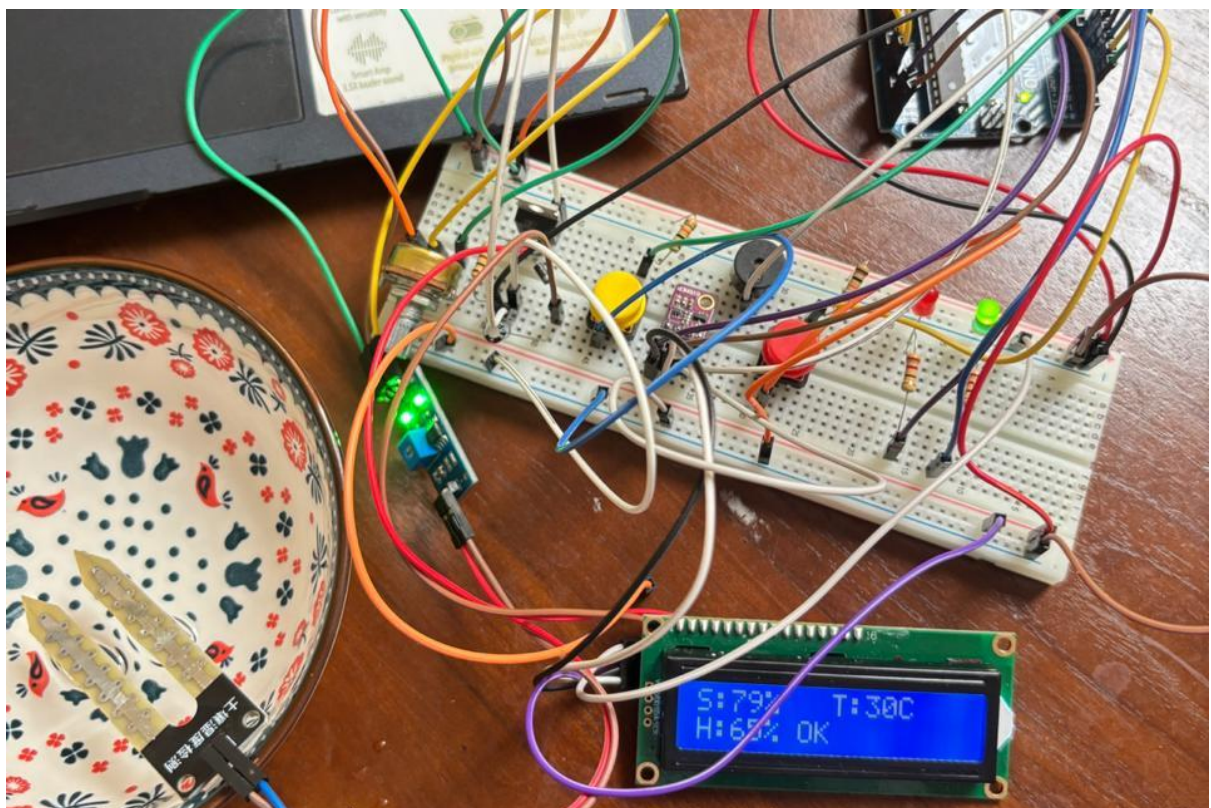


Figure B2. Smart Plant System in Operation — Adequate Soil Moisture Condition, Pump Standby Mode



Figure B3. USART Serial Monitor Telemetry Output During Live Testing

APPENDIX C: AVR REGISTER CONFIGURATION SUMMARY

Register	Value (Binary / Hex)	Module	Function
ADMUX	0b01000000 / 0x40	ADC (Module 8)	AVcc reference, right-adjust result, channel 0 (A0)
ADCSRA	0b10000111 / 0x87	ADC (Module 8)	ADC enable, prescaler 128 (125 kHz ADC clock)
TCCR0A	0b10000011 / 0x83	Timer0 PWM (Module 7)	Fast PWM mode, non-inverting OC0A output, WGM01+WGM00
TCCR0B	0b00000011 / 0x03	Timer0 PWM (Module 7)	Fast PWM WGM02=0, prescaler /64, ~977 Hz PWM frequency
OCR0A	0–120 (variable)	Timer0 PWM (Module 7)	Pump PWM duty cycle (0=off, 120=~47% max)

TCCR1A	0b00000000 / 0x00	Timer1 CTC (Module 5)	Normal operation on OC1A/B, WGM11+WGM10 = 00
TCCR1B	0b00001011 / 0x0B	Timer1 CTC (Module 5)	CTC mode (WGM12=1), prescaler /64
OCR1AL/H	249 / 0x00F9	Timer1 CTC (Module 5)	Compare match at 249 → 1 ms interrupt at 16 MHz, /64
TIMSK1	0b00000010 / 0x02	Timer1 / ISR (Module 5/6)	Enable OCIE1A: Timer1 COMPA interrupt
UCSR0B	0b00001000 / 0x08	USART (Module 3)	Enable TX only (TXEN0), no RX
UBRR0L/H	103 / 0x0067	USART (Module 3)	9600 baud at 16 MHz: $UBRR = F_CPU/16/\text{baud} - 1$
TWBR	72 / 0x48	I2C TWI (Module 9)	SCL frequency ≈ 100 kHz at 16 MHz, prescaler = 1
TWSR	0b00000000 / 0x00	I2C TWI (Module 9)	TWI prescaler bits = 00 (prescaler value = 1)
DDRD	0b11110000 / 0xF0 (partial)	I/O (Module 2)	PD4,5,6,7 = OUTPUT; PD2,3 = INPUT (cleared)
PORTD	0b00001100 / 0x0C (partial)	I/O (Module 2)	PD2,3 pull-up enabled (INPUT_PULLUP)
EECR	EEMPE+EEPE sequence	EEPROM (Module 7)	Write enable: set EEMPE, then EEPE within 4 cycles

Table C1. Complete AVR Register Configuration Summary

APPENDIX D: DECISION LOGIC STATE TABLE

Soil %	Temperature	Humidity	pump_state	PWM Value	LED	Buzzer
< 20% (Critical)	Any	Any	ON	120 (+ any boost)	Red	ON — Critical alert

20–34%	> 30°C	< 40%	ON	110 (100 + boost)	Red	OFF
20–34%	<= 30°C	>= 40%	ON	100	Red	OFF
35% to threshold	> 30°C	< 40%	ON	90 (80 + boost)	Red	OFF
35% to threshold	<= 30°C	>= 40%	ON	80	Red	OFF
> threshold	Any	Any	OFF	0	Green	OFF
> threshold	Any	> 85%	OFF	0	Green	ON — Humidity alert
Any (Manual)	Any	Any	FORCED ON	120	Red	OFF

Table D1. Decision Logic State Table — All Operating Conditions